DSA Programs 12–25

12. Stack - Infix to Postfix

Input: A+B*C Output: ABC*+

```
#include<stdio.h>
#include<ctype.h>
char stack[100];
int top=-1;
void push(char x){
   stack[++top]=x;
}

char pop(){
   return stack[top--];
}

int priority(char x){
   if(x=='+'||x=='-') return 1;
   if(x=='*'||x=='/') return 2;
   return 0;
}

int main(){
   char exp[100], *e, x;

   scanf("%s", exp);
```

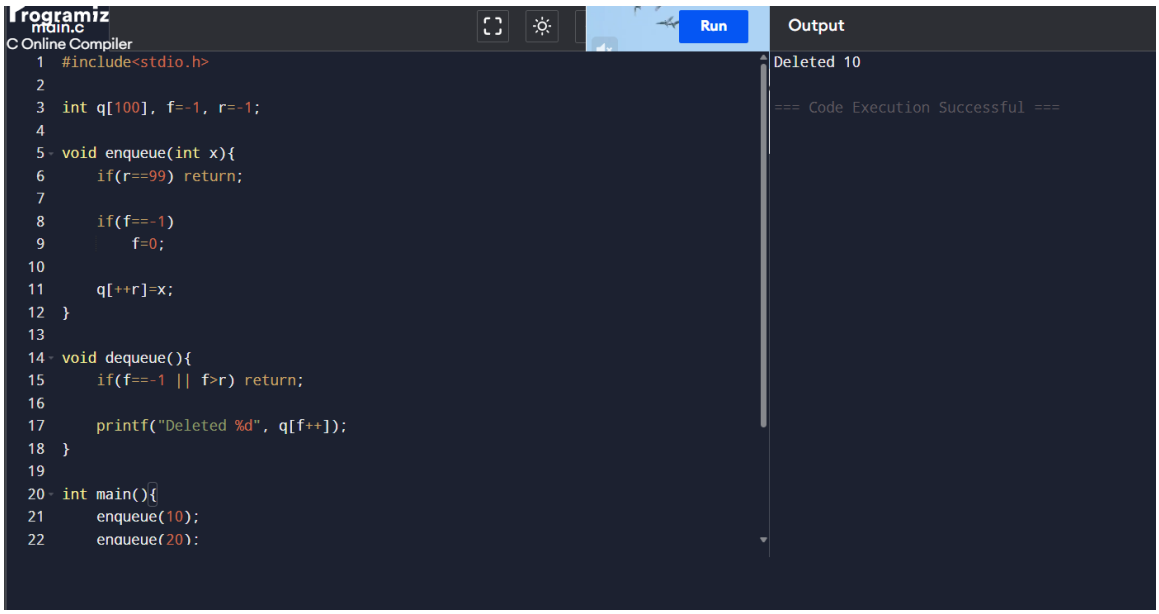
```
    e=exp;

    while(*e){
        if(isalnum(*e))
            printf("%c", *e);
        else if(*e=='(')
            push(*e);
        else if(*e==')'){
            while((x=pop())!='(')
                printf("%c", x);
        }
        else{
            while(top!=-1 && priority(stack[top])>=priority(*e))
                printf("%c", pop());
            push(*e);
        }
        e++;
    }

    while(top!=-1)
        printf("%c", pop());

    return 0;
}
```

13. Queue using Array

Input: 10,20,30 Output: Deleted 10

```
#include<stdio.h>

int q[100], f=-1, r=-1;
void enqueue(int x){
   if(r==99) return;

   if(f==-1)
      f=0;
   q[++r]=x;
}
void dequeue(){
   if(f==-1 || f>r) return;

   printf("Deleted %d", q[f++]);
}
int main(){
   enqueue(10);
   enqueue(20);
   enqueue(30);

   dequeue();
```

return 0;



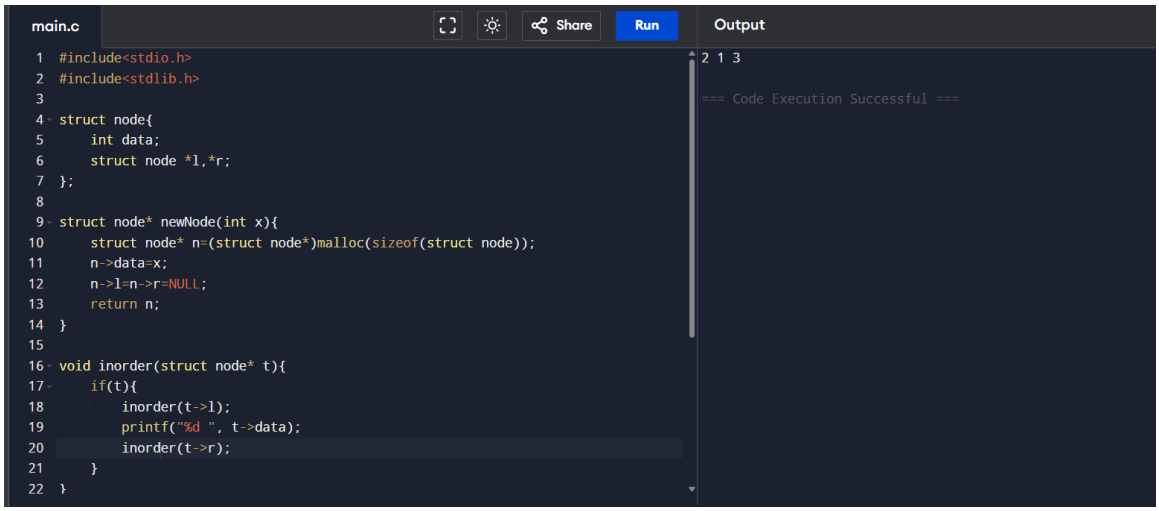
}

14. Tree Traversal

Input: Binary Tree Output: Inorder sequence

```
#include<stdio.h>

#include<stdlib.h>

struct node{

   int data;

   struct node *l,*r;

};

struct node* newNode(int x){

   struct node* n=(struct node*)malloc(sizeof(struct node));

   n->data=x;

   n->l=n->r=NULL;

   return n;

}
```

```
void inorder(struct node* t){
   if(t){
      inorder(t->l);
      printf("%d ", t->data);
      inorder(t->r);
   }
}
int main(){
   struct node* root=newNode(1);
   root->l=newNode(2);
   root->r=newNode(3);
   inorder(root);
   return 0;
}
```



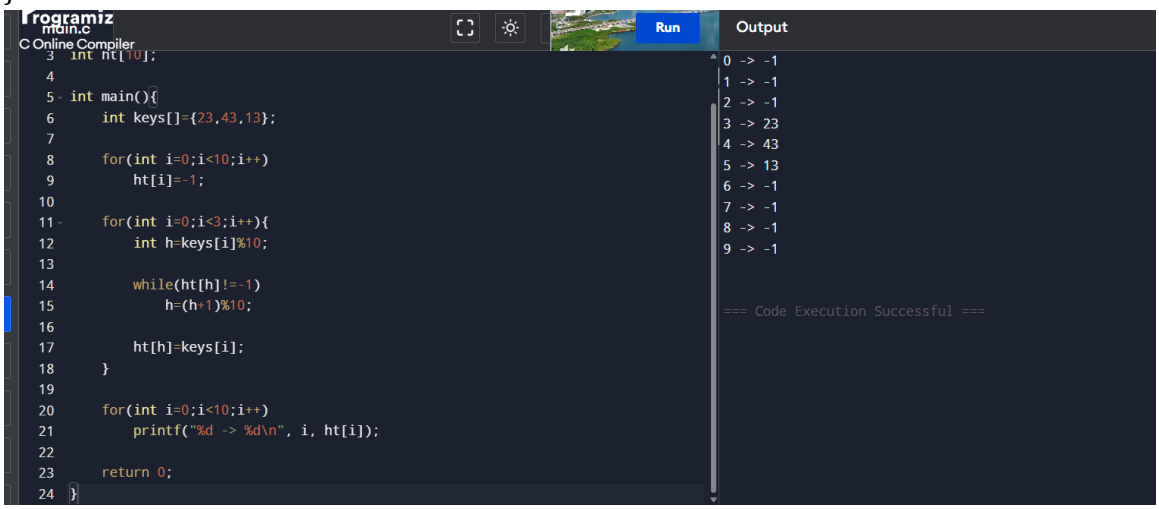
15. Hashing Linear Probing

Input: 23,43,13 Output: Hash Table

```
#include<stdio.h>
```

```
int ht[10];

int main(){
   int keys[]={23,43,13};

   for(int i=0;i<10;i++)
      ht[i]=-1;

   for(int i=0;i<3;i++){
      int h=keys[i]%10;

      while(ht[h]!=-1)
         h=(h+1)%10;

      ht[h]=keys[i];
   }

   for(int i=0;i<10;i++)
      printf("%d -> %d\n", i, ht[i]);

   return 0;
```

}

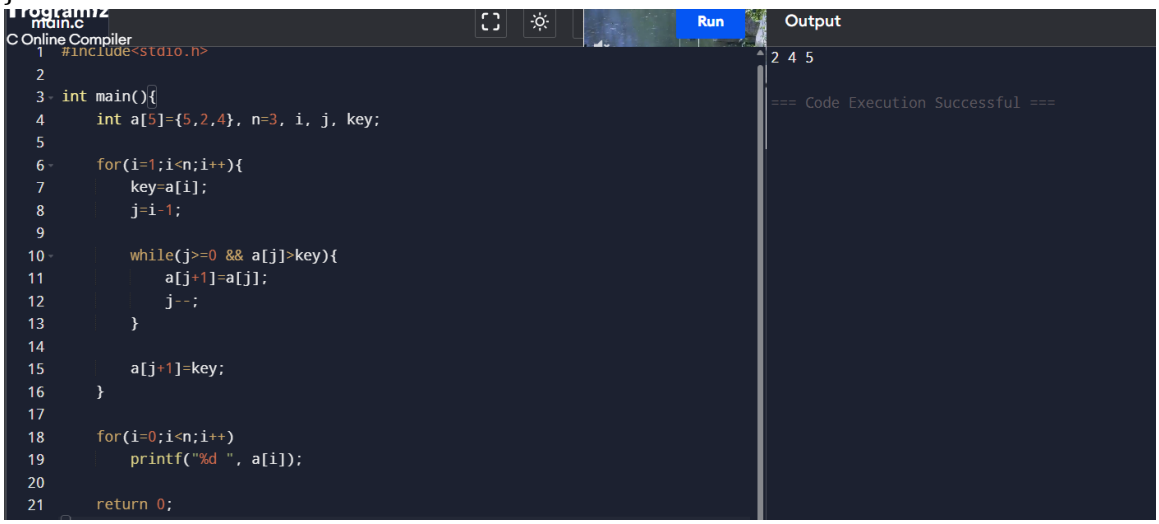


16. Insertion Sort

Input: 5 2 4 Output: 2 4 5

```
#include<stdio.h>


int main(){
   int a[5]={5,2,4}, n=3, i, j, key;


   for(i=1;i<n;i++){
      key=a[i];
      j=i-1;


      while(j>=0 && a[j]>key){
         a[j+1]=a[j];
         j--;
      }


      a[j+1]=key;
```

```
}

for(i=0;i<n;i++)
   printf("%d ", a[i]);

return 0;
}
```

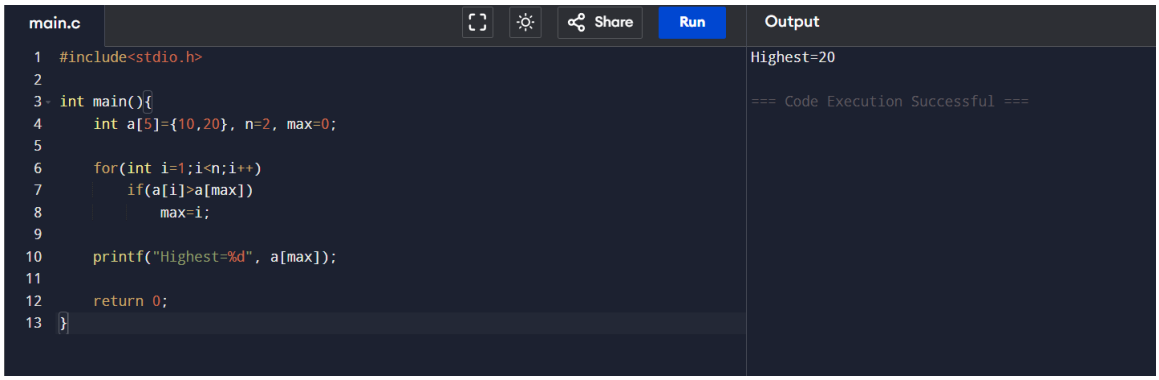


17. Priority Queue

Input: 10,20 Output: Highest priority element#include<stdio.h>

```
int main(){
   int a[5]={10,20}, n=2, max=0;

   for(int i=1;i<n;i++)
      if(a[i]>a[max])
         max=i;
```

```
    printf("Highest=%d", a[max]);


    return 0;
}
```



18. Merge Sort

Input: 3 1 Output: 1 3

```
#include<stdio.h>


void merge(int a[], int l, int m, int r){
   int i=l, j=m+1, k=0, b[100];


   while(i<=m && j<=r)
      b[k++]=(a[i]<a[j])?a[i++]:a[j++];


   while(i<=m)
      b[k++]=a[i++];


   while(j<=r)
      b[k++]=a[j++];
```

```
   for(i=l,k=0;i<=r;i++,k++)
      a[i]=b[k];
}

void ms(int a[], int l, int r){
   if(l<r){
      int m=(l+r)/2;

      ms(a,l,m);
      ms(a,m+1,r);
      merge(a,l,m,r);
   }
}

int main(){
   int a[]={3,1};

   ms(a,0,1);

   printf("%d %d", a[0], a[1]);

   return 0;
```

}

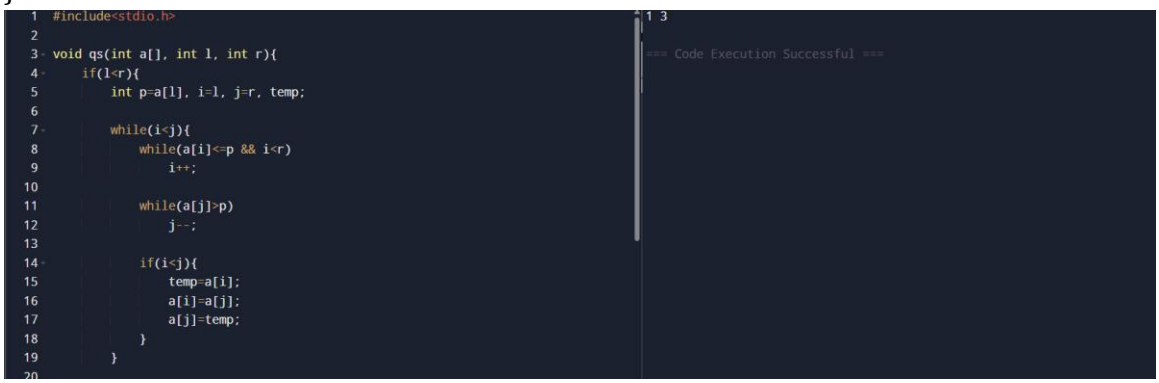
19. Quick Sort

Input: 3 1 Output: 1 3

```
#include<stdio.h>


void qs(int a[], int l, int r){
  if(l<r){
    int p=a[l], i=l, j=r, temp;


    while(i<j){
      while(a[i]<=p && i<r)
        i++;


      while(a[j]>p)
        j--;


      if(i<j){
        temp=a[i];
```

```
            a[i]=a[j];
            a[j]=temp;
          }
        }

        temp=a[l];
        a[l]=a[j];
        a[j]=temp;

        qs(a,l,j-1);
        qs(a,j+1,r);
    }
}

int main(){
    int a[]={3,1};

    qs(a,0,1);

    printf("%d %d", a[0], a[1]);

    return 0;
```

}



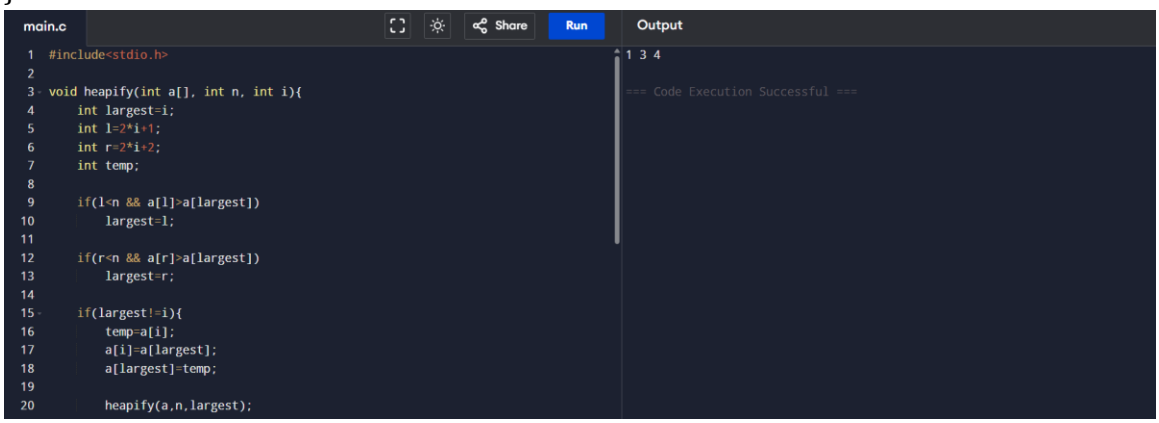
20. Heap Sort

Input: 4 1 3 Output: 1 3 4

```
#include<stdio.h>


void heapify(int a[], int n, int i){
   int largest=i;
   int l=2*i+1;
   int r=2*i+2;
   int temp;

   if(l<n && a[l]>a[largest])
      largest=l;

   if(r<n && a[r]>a[largest])
      largest=r;

   if(largest!=i){
      temp=a[i];
      a[i]=a[largest];
```

```
        a[largest]=temp;

        heapify(a,n,largest);
    }
}

int main(){
    int a[]={4,1,3};
    int n=3;
    int temp;

    for(int i=n/2-1;i>=0;i--)
        heapify(a,n,i);

    for(int i=n-1;i>0;i--){
        temp=a[0];
        a[0]=a[i];
        a[i]=temp;

        heapify(a,i,0);
    }

    for(int i=0;i<n;i++)
        printf("%d ",a[i]);

    return 0;
```

}




```c
#include<stdio.h>

void heapify(int a[], int n, int i){
    int largest=i;
    int l=2*i+1;
    int r=2*i+2;
    int temp;

    if(l<n && a[l]>a[largest])
        largest=l;

    if(r<n && a[r]>a[largest])
        largest=r;

    if(largest!=i){
        temp=a[i];
        a[i]=a[largest];
        a[largest]=temp;

        heapify(a,n,largest);
```

21. AVL Tree (Insertion)

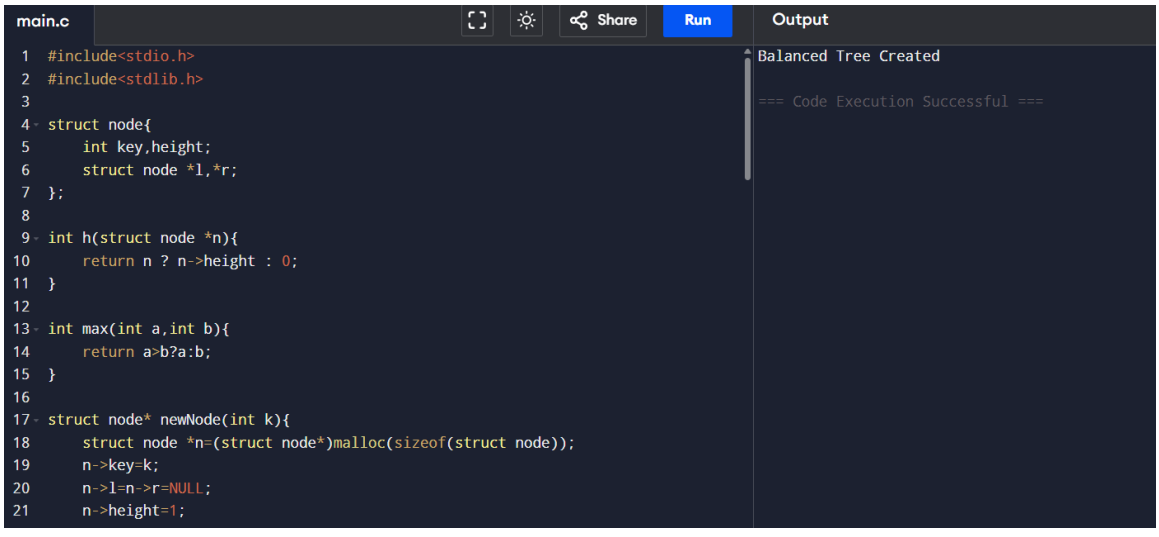
Input: 10 20 30 Output: Balanced Tree

```c
#include<stdio.h>

#include<stdlib.h>


struct node{

   int key,height;

   struct node *l,*r;

};


int h(struct node *n){

   return n ? n->height : 0;

}


int max(int a,int b){

   return a>b?a:b;

}
```

```
struct node* newNode(int k){
   struct node *n=(struct node*)malloc(sizeof(struct node));
   n->key=k;
   n->l=n->r=NULL;
   n->height=1;
   return n;
}

struct node* rr(struct node *y){
   struct node *x=y->l;
   y->l=x->r;
   x->r=y;

   y->height=max(h(y->l),h(y->r))+1;
   x->height=max(h(x->l),h(x->r))+1;

   return x;
}

struct node* lr(struct node *x){
   struct node *y=x->r;
   x->r=y->l;
   y->l=x;

   x->height=max(h(x->l),h(x->r))+1;
   y->height=max(h(y->l),h(y->r))+1;
```

```
    return y;
}

int bf(struct node *n){
    return h(n->l)-h(n->r);
}

struct node* insert(struct node *n,int k){
    if(!n)
        return newNode(k);

    if(k<n->key)
        n->l=insert(n->l,k);
    else
        n->r=insert(n->r,k);

    n->height=1+max(h(n->l),h(n->r));

    int b=bf(n);

    if(b>1 && k<n->l->key)
        return rr(n);

    if(b<-1 && k>n->r->key)
        return lr(n);
```

```
    return n;
}

int main(){
    struct node *root=NULL;

    root=insert(root,10);
    root=insert(root,20);
    root=insert(root,30);

    printf("Balanced Tree Created");

    return 0;
}
```

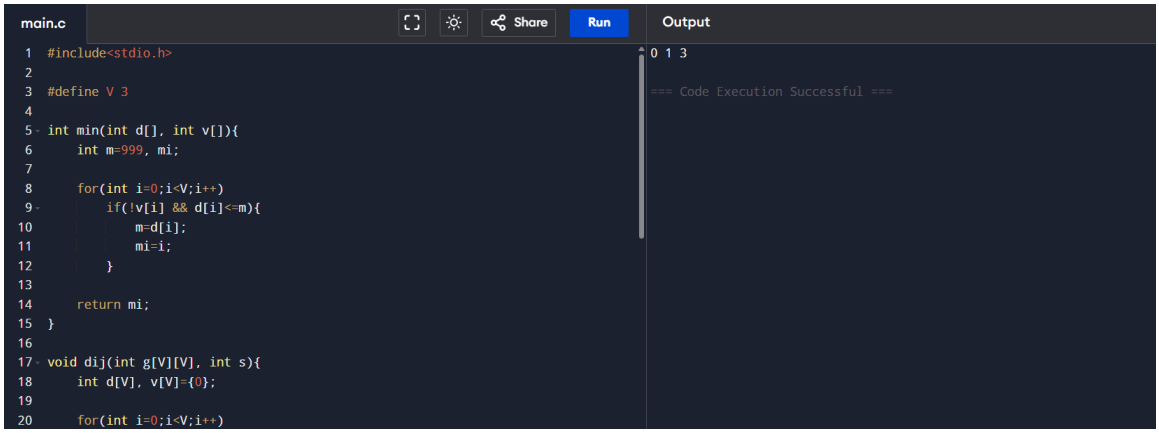


22. Dijkstra Algorithm

Input: Graph Output: Shortest Path

```
#include<stdio.h>

#define V 3

int min(int d[], int v[]){
   int m=999, mi;

   for(int i=0;i<V;i++)
      if(!v[i] && d[i]<=m){
         m=d[i];
         mi=i;
      }

   return mi;
}

void dij(int g[V][V], int s){
   int d[V], v[V]={0};

   for(int i=0;i<V;i++)
      d[i]=999;

   d[s]=0;
```

```
    for(int c=0;c<V-1;c++){
        int u=min(d,v);
        v[u]=1;

        for(int i=0;i<V;i++)
            if(!v[i] && g[u][i] &&
               d[u]+g[u][i]<d[i])
                d[i]=d[u]+g[u][i];
    }

    for(int i=0;i<V;i++)
        printf("%d ",d[i]);
}

int main(){
    int g[V][V]={{0,1,4},
                 {1,0,2},
                 {4,2,0}};

    dij(g,0);

    return 0;
}
```

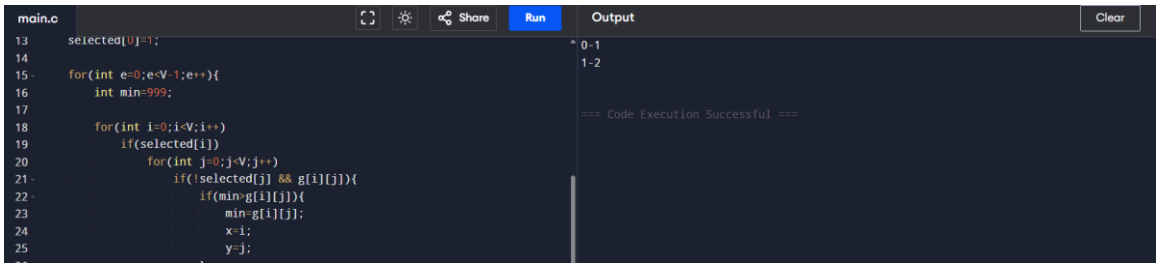


23. Prim's Algorithm

Input: Graph Output: MST

```
#include<stdio.h>


#define V 3


int main(){
  int g[V][V]={{0,1,4},
               {1,0,2},
               {4,2,0}};


  int selected[V]={0};
  int x,y;


  selected[0]=1;


  for(int e=0;e<V-1;e++){
    int min=999;
```

```
    for(int i=0;i<V;i++)
      if(selected[i])
        for(int j=0;j<V;j++)
          if(!selected[j] && g[i][j]){
            if(min>g[i][j]){
              min=g[i][j];
              x=i;
              y=j;
            }
          }

    printf("%d-%d\n",x,y);
    selected[y]=1;
  }

  return 0;
}
```



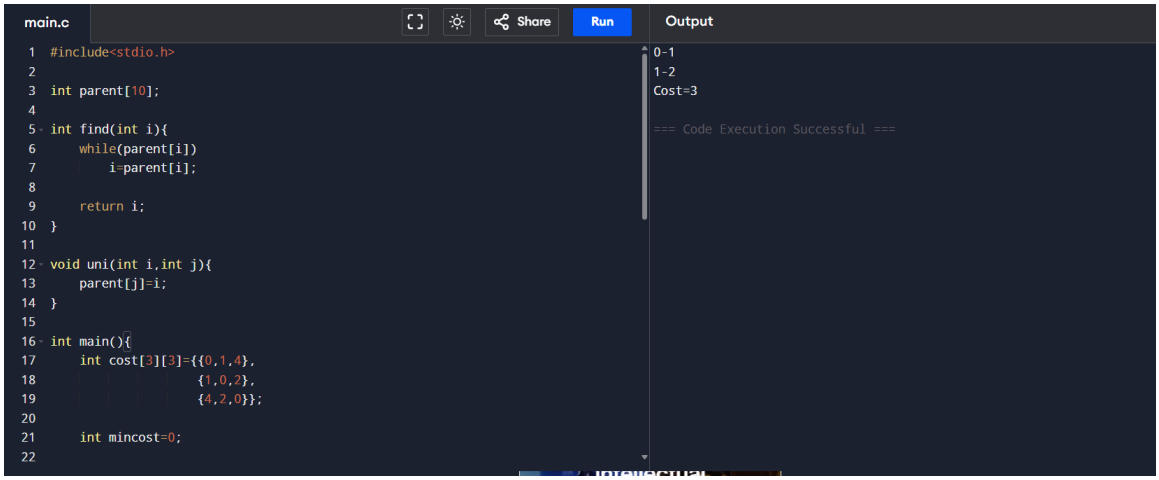
24. Kruskal Algorithm

Input: Edges Output: MST

```
#include<stdio.h>

int parent[10];

int find(int i){
   while(parent[i])
      i=parent[i];

   return i;
}

void uni(int i,int j){
   parent[j]=i;
}

int main(){
   int cost[3][3]={{0,1,4},
                   {1,0,2},
                   {4,2,0}};

   int mincost=0;

   for(int i=0;i<3;i++)
      parent[i]=0;
```

```
for(int e=0;e<2;e++){
   int min=999,a,b;

   for(int i=0;i<3;i++)
      for(int j=0;j<3;j++)
         if(cost[i][j]<min && cost[i][j]!=0){
            min=cost[i][j];
            a=i;
            b=j;
         }

   int u=find(a);
   int v=find(b);

   if(u!=v){
      uni(u,v);
      printf("%d-%d\n",a,b);
      mincost+=min;
   }

   cost[a][b]=cost[b][a]=999;
}

printf("Cost=%d",mincost);
```

```
    return 0;
}
```



25. Hashing using linear probing

```
#include <stdio.h>


#define SIZE 10


int main() {
   int hashTable[SIZE];
   int keys[] = {23, 43, 13, 27, 33};
   int n = 5;


   for(int i = 0; i < SIZE; i++)
      hashTable[i] = -1;


   for(int i = 0; i < n; i++) {
      int key = keys[i];
```

```
    int index = key % SIZE;

    while(hashTable[index] != -1)
        index = (index + 1) % SIZE;

    hashTable[index] = key;
  }

  printf("Hash Table:\n");
  for(int i = 0; i < SIZE; i++) {
    if(hashTable[i] != -1)
        printf("%d --> %d\n", i, hashTable[i]);
    else
        printf("%d --> Empty\n", i);
  }

  return 0;
}
```